Module 05 — File Handling

Phase: Advanced Python
Previous: Exception Handling
Next: Logging

Module Outcome

Save and reload text on disk with `with open`, choose the right mode, recover from a missing file, and use CSV for tables and JSON for config.

By the end of this module you will be able to:

- Explain why RAM forgets and files remember
- Open text files with `with open(..., encoding="utf-8")`
- Choose `r` / `w` / `a` and not wipe a file by accident
- Read and write text; iterate lines on larger files
- Catch `FileNotFoundError` on a missing path
- Write and read a CSV table and a JSON config

1. Why This Concept Matters

Variables live in **RAM**. When the program ends, they are gone. A month of expenses cannot live only in a list.

A **file** is data on disk that survives a restart. You open it, read or write, and close it.

```
TEXT                            BINARY
----------------                ----------------
Characters                      Bytes
.txt .csv .json .log            images, zip, pickle
encoding="utf-8"                no encoding
modes: r / w / a                modes: rb / wb / ab
```

Tonight's work is **text**. Do not open a `.jpg` in text mode. Binary is a short awareness note at the end.

A **path** is the location of the file — usually a string such as `"data/expenses.txt"`. Python looks relative to the folder you ran the script from. Create folders first (`os.makedirs("data", exist_ok=True)`) so `open` is not asked to write into a missing directory.

2. `open()` and `with open()`

Understand

Before you can read or write, you **open** the file. `open` gives you a file object. When you are done, you **close** it so data is flushed and the handle is released.

`with open(...) as f:` closes the file automatically — even if an exception is raised. That is the everyday form.

How It Works

```
import os

os.makedirs("data", exist_ok=True) # this creates data folder in current location if not exist

f = open("data/expenses.txt", "w", encoding="utf-8")
f.write("2026-09-04 | food | 120 | Tea\n")
f.close()
```

Prefer `with` so you cannot forget `close()`:

```
import os

os.makedirs("data", exist_ok=True)

with open("data/expenses.txt", "w", encoding="utf-8") as f:
    f.write("2026-09-04 | food | 120 | Tea\n")

with open("data/expenses.txt", "r", encoding="utf-8") as f:
    print(f.read().strip())
```

Expected Output:

```
2026-09-04 | food | 120 | Tea
```

Always set `encoding="utf-8"` for text.

⚠️ WATCH OUT

```
# Wrong — if write() raises, close() never runs
f = open("data/expenses.txt", "w", encoding="utf-8")
f.write(line)
f.close()

# Prefer
with open("data/expenses.txt", "w", encoding="utf-8") as f:
    f.write(line)
```

Try It

Create `data/` if needed. Write `"hello\n"` to `data/note.txt` with `with` and mode `"w"`. Open it again and print `read()`.

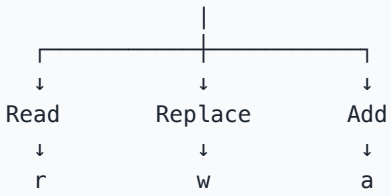
---

3. File modes — read, replace, append

Understand

The **mode** is *why* you opened the file. The wrong mode silently wipes a month of expenses (`w`) or refuses to open (`r` on a missing file).

```
What do I want?
                    File
```



Mode	Plain English	Missing file?	Existing content?
`r`	**Read** what is already there	`FileNotFoundError`	Left as-is
`w`	**Create / replace**	Creates it	**Cleared** (truncated)
`a`	**Add** without deleting old lines	Creates it	Kept; new writes go at the **end**

`r` is the default if you omit the mode. Other modes (`r+`, `w+`, `a+`, `x`) are named in Advanced. You do not need them for the tracker.

How It Works

```
import os

os.makedirs("data", exist_ok=True)

with open("data/ward.txt", "w", encoding="utf-8") as f:
    f.write("Bed 12: stable\n")

with open("data/ward.txt", "a", encoding="utf-8") as f:
    f.write("Bed 14: observation\n")

with open("data/ward.txt", "r", encoding="utf-8") as f:
    print(f.read())
```

Expected Output:

```
Bed 12: stable
Bed 14: observation
```

⚠ WATCH OUT

```
# Wrong — "w" empties the file before you write
with open("data/expenses.txt", "w", encoding="utf-8") as f:
    f.write("new line\n")     # previous month is gone

# Prefer when you mean "add a line"
with open("data/expenses.txt", "a", encoding="utf-8") as f:
    f.write("new line\n")
```

`w` is a **mode** (how you opened). `write()` is a **method** (the call that sends text). Opening with `w` does not write data by itself — but it has already cleared the file.

Try It

Write `"Bed 12: stable\n"` with `"w"`, then append `"Bed 14: observation\n"` with `"a"`. Read and print. If you had used `"w"` the second time, what would be missing?

---

4. Reading text

Understand

After you open for read, you pull characters (or lines) from the file.

Method	Returns	When to use
`read()`	Entire file as **one string**	Small files
`readline()`	**One** line	Line-by-line protocols
`readlines()`	**List** of all lines	Medium files — whole list in RAM
`for line in f`	One line at a time	**Large files** — this is the habit

A 2 GB log must not be loaded with `read()`. Iterate.

The file pointer moves as you read. A second `read()` without reopening often returns empty.

How It Works

```
import os

os.makedirs("data", exist_ok=True)
lines = [
    "2026-09-04 | food | 120 | Tea",
    "2026-09-04 | travel | 50 | Metro",
    "2026-09-04 | food | 40 | Samosa",
]

with open("data/expenses.txt", "w", encoding="utf-8") as f:
    for line in lines:
        f.write(line + "\n")

with open("data/expenses.txt", "r", encoding="utf-8") as f:
    for line in f:
        print(line.strip())
```

Expected Output:

```
2026-09-04 | food | 120 | Tea
2026-09-04 | travel | 50 | Metro
2026-09-04 | food | 40 | Samosa
```

Same small file: `read()` (whole string) vs iterating lines.

```
import os

os.makedirs("data", exist_ok=True)
```

```
with open("data/expenses.txt", "w", encoding="utf-8") as f:
    f.write("Tea\nMetro\nSamosa\n")

with open("data/expenses.txt", "r", encoding="utf-8") as f:
    all_text = f.read()
    print("read():", repr(all_text))

with open("data/expenses.txt", "r", encoding="utf-8") as f:
    for line in f:
        print("line:", line.strip())
```

Expected Output:

```
read(): 'Tea\nMetro\nSamosa\n'
line: Tea
line: Metro
line: Samosa
```

⚠️ WATCH OUT

```
# Wrong for a huge log — builds one giant string in RAM
text = open("app.log", encoding="utf-8").read()

# Prefer
with open("app.log", encoding="utf-8") as f:
    for line in f:
        ...
```

`readlines()` also builds a **list in memory**. `for line in f` yields **one line at a time**.

Try It

Write three product names (one per line) to `products.txt`. Count lines **without** `readlines()`. Print the count.

---

5. Missing files — `FileNotFoundError`

Understand

A path might not exist yet. `open(..., "r")` then raises `FileNotFoundError`. A missing CSV should print a clear message, not a traceback.

This is last module's exception, used on purpose.

`"w"` / `"a"` *create* a missing file — you do not get `FileNotFoundError` in those modes.

How It Works

```
import os

os.makedirs("data", exist_ok=True)

with open("data/expenses.txt", "w", encoding="utf-8") as f:
    f.write("2026-09-04 | food | 120 | Tea\n")

def read_text(path):
    try:
```

```
        with open(path, "r", encoding="utf-8") as f:
            return f.read()
    except FileNotFoundError:
        print(f"File not found: {path}")
        return None

for fname in ("data/expenses.txt", "data/missing.txt"):
    print("Checking:", fname)
    data = read_text(fname)
    if data is not None:
        print(data.strip())
```

Expected Output:

```
Checking: data/expenses.txt
2026-09-04 | food | 120 | Tea
Checking: data/missing.txt
File not found: data/missing.txt
```

⚠ WATCH OUT

Do not skip handling. Checking `os.path.exists` first is valid (Look Before You Leap). After Exceptions week, `try` / `except FileNotFoundError` is the usual Python style.

Try It

Try to read `data/no_such.txt`. Catch `FileNotFoundError` and print a one-line message. Do not let a traceback reach the screen.

---

6. CSV — when the data is a table

Understand

CSV means **Comma Separated Values** — a text table Excel and Sheets understand. Free text (`Tea | 120`) is fine for two lines. A month of expenses needs **columns**: date, category, amount, note.

- `csv.writer` / `csv.reader` — rows as lists
- `csv.DictWriter` / `csv.DictReader` — rows as dicts keyed by the header (prefer this when columns have names)

Open CSV with `newline=""` (Python 3 / Windows extra blank lines otherwise). Amounts in CSV are **strings** until you convert: `int(row["amount"])`.

How It Works

```
import csv
import os

os.makedirs("data", exist_ok=True)

rows = [
    {"date": "2026-09-04", "category": "food", "amount": "120", "note": "Tea"},
    {"date": "2026-09-04", "category": "travel", "amount": "50", "note": "Metro"},
    {"date": "2026-09-04", "category": "food", "amount": "40", "note": "Samosa"},
]
```

```
with open("data/expenses.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["date", "category", "amount", "note"])
    writer.writeheader()
    writer.writerows(rows)

total = 0
with open("data/expenses.csv", newline="", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        print(row["note"], "₹" + row["amount"])
        total += int(row["amount"])
print("Total:", total)
```

Expected Output:

```
Tea ₹120
Metro ₹50
Samosa ₹40
Total: 210
```

⚠️ WATCH OUT

```
# Wrong — amounts are strings until you convert
total += row["amount"]          # TypeError or string concat

# Prefer
total += int(row["amount"])
```

Do not open CSV as `"wb"`. `DictReader` uses the header row as keys. `reader` gives lists — you remember column indexes yourself.

Try It

Write two rows to `data/items.csv` with columns `name` and `price`. Read them back with `DictReader` and print each `name` and `price`.

---

7. JSON — when the data is config

Understand

JSON is nested data as text — the language of APIs and config files. Categories, daily limit, and currency belong in `data/config.json`, not hardcoded in the middle of `main`.

Function	Reads / writes
`json.dump(obj, file)`	Object → **file**
`json.load(file)`	**File** → object
`json.dumps(obj)`	Object → **string** (`s` = string)
`json.loads(s)`	**String** → object

```
Is my data basically a table?
        YES  →  CSV   (Excel / report / import / export)
```

```
Is my data configuration or a nested structure?
        YES  →  JSON  (API / config / preferences)
```

How It Works

```
import json
import os

os.makedirs("data", exist_ok=True)

config = {
    "app_name": "ExpenseTracker",
    "currency": "INR",
    "max_daily_limit": 5000,
    "categories": ["food", "travel", "rent"],
}

with open("data/config.json", "w", encoding="utf-8") as f:
    json.dump(config, f, indent=2)

with open("data/config.json", "r", encoding="utf-8") as f:
    loaded = json.load(f)
    print("Daily limit:", loaded["max_daily_limit"])
    print("Categories:", loaded["categories"])
```

Expected Output:

```
Daily limit: 5000
Categories: ['food', 'travel', 'rent']
```

⚠️ WATCH OUT

`json.load` → file. `json.loads` → string. The extra **s** is “string”. Mixing them is a common crash (`str` has no `.read()`).

Try It

Write `{"theme": "light", "lang": "en"}` to `prefs.json`. Load it, set `theme` to `"dark"`, save again, and print the loaded dict.

---

8. Advanced file handling (awareness)

You do **not** need this section to finish the expense tracker. Skim once.

Other modes. `r+` read-and-write (file must exist, no truncate). `w+` write-and-read (**truncates** like `w`). `a+` append-and-read. `x` create exclusive — error if the file exists. Suffix `b` for binary. If you are unsure, you still only need `r`, `w`, and `a`.

Cursor. `tell()` reports the current index (0-based). `seek(offset)` moves there. In text mode, prefer an offset that came from `tell()`.

Binary. Open with `rb` / `wb` / `ab`. Do **not** pass `encoding`. Copy bytes: read from one file, write to another. Do not use text mode for a `.jpg`.

Pickle. `pickle.dump` / `pickle.load` save a Python object as bytes (`wb` / `rb`). **Do not unpickle data you do not trust — pickle can run code.** For tables and config you share with humans or other languages, prefer JSON or CSV.

Directories. `os.makedirs(path, exist_ok=True)` creates nested folders. `os.listdir(path)` lists **one** folder. `os.walk(path)` visits that folder and every subdirectory.

Step-by-Step: Build It

One assemble: folders, `with`, UTF-8, CSV `newline=""`, `DictWriter` / `DictReader`, JSON config. Logging to `logs/app.log` comes in the next module.

```
import csv
import json
import os

os.makedirs("data", exist_ok=True)

config = {
    "app_name": "ExpenseTracker",
    "currency": "INR",
    "max_daily_limit": 5000,
    "categories": ["food", "travel", "rent"],
}
with open("data/config.json", "w", encoding="utf-8") as f:
    json.dump(config, f, indent=2)

rows = [
    {"date": "2026-09-04", "category": "food", "amount": "120", "note": "Tea"},
    {"date": "2026-09-04", "category": "travel", "amount": "50", "note": "Metro"},
]
with open("data/expenses.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["date", "category", "amount", "note"])
    writer.writeheader()
    writer.writerows(rows)
print("Wrote data/expenses.csv")

total = 0
with open("data/expenses.csv", newline="", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        print(f"{row['note']}: ₹{row['amount']}")
        total += int(row["amount"])

with open("data/config.json", "r", encoding="utf-8") as f:
    limit = json.load(f)["max_daily_limit"]

print("Day total ₹%s (limit ₹%s)" % (total, limit))
```

Expected Output:

```
Wrote data/expenses.csv
Tea: ₹120
Metro: ₹50
Day total ₹170 (limit ₹5000)
```

Then open the kit: 01_python_expense_tracker — add expenses, export CSV. Do not copy `solution/`. Wire logging in Logging. Stretch: 15_generator_csv_stream — wire `yield` to `csv.DictReader`.

Practice Exercises

Attempt each problem before reading the model answer.

1. Write `"Bed 12: stable\n"` to `ward.txt` with `w`, then **append** `"Bed 14: observation\n"` with `a`. Read and print the file.

Hint: A second `"w"` would replace the file — Bed 12 would be gone.

Model answer:

```
with open("ward.txt", "w", encoding="utf-8") as f:
    f.write("Bed 12: stable\n")
with open("ward.txt", "a", encoding="utf-8") as f:
    f.write("Bed 14: observation\n")
with open("ward.txt", "r", encoding="utf-8") as f:
    print(f.read())
```

Expected Output:

```
Bed 12: stable
Bed 14: observation
```

1. Copy `source.txt` to `backup.txt` using `with open` on both files. (Create `source.txt` first with one line of text.)

Model answer:

```
with open("source.txt", "w", encoding="utf-8") as f:
    f.write("keep me\n")

with open("source.txt", encoding="utf-8") as src, open("backup.txt", "w", encoding="utf-8") as dst:
    dst.write(src.read())

with open("backup.txt", encoding="utf-8") as f:
    print(f.read())
```

Expected Output:

```
keep me
```

1. `products.txt` has one product name per line. Count non-blank lines **without** `readlines()`. Print the count.

Model answer:

```
with open("products.txt", "w", encoding="utf-8") as f:
    f.write("Tea\nSamosa\n\nIdli\n")

count = 0
with open("products.txt", encoding="utf-8") as f:
    for line in f:
        if line.strip():
```

```
            count += 1
print(count)
```

Expected Output:

```
3
```

1. Read `appointments.csv` with columns `patient` and `fee`. Print the total fee. If the file is missing, print a clear message instead of crashing.

Model answer:

```
import csv

try:
    total = 0
    with open("appointments.csv", newline="", encoding="utf-8") as f:
        for row in csv.DictReader(f):
            total += int(row["fee"])
    print("Total fees:", total)
except FileNotFoundError:
    print("appointments.csv not found")
```

Expected Output (when the file is missing):

```
appointments.csv not found
```

1. Load or create preferences in `prefs.json`. If the file exists, load it; otherwise start from `{"theme": "light", "lang": "en"}`. Set `theme` to `"dark"` and save. Print the dict.

Model answer:

```
import json

prefs = {"theme": "light", "lang": "en"}
try:
    with open("prefs.json", "r", encoding="utf-8") as f:
        prefs = json.load(f)
except FileNotFoundError:
    pass
prefs["theme"] = "dark"
with open("prefs.json", "w", encoding="utf-8") as f:
    json.dump(prefs, f, indent=2)
print(prefs)
```

Expected Output (first run, file missing):

```
{'theme': 'dark', 'lang': 'en'}
```

Lab

Combine `with`, modes `w` / `a` / `r`, a missing-file handler, and a tiny CSV. This is not the full expense-tracker kit.

Problem

A clerk keeps a short ward log and a two-column fee table. Restarting Python must not wipe the log. A missing CSV must not dump a traceback.

What to Build

A short script that writes a log, appends a second line, reads it back, then writes and totals a CSV — with a safe read for a path that does not exist.

Requirements

- `os.makedirs("data", exist_ok=True)`
- Write `data/ward.txt` with `"Bed 12: stable\n"`, then **append** `"Bed 14: observation\n"`
- Print the full log
- Write `data/fees.csv` with header `patient,fee` and two rows
- Read with `csv.DictReader` and print `Total: <sum>`
- Try to read `data/missing.csv`; catch `FileNotFoundError` and print `No file yet`

Expected Behavior

The log contains **both** beds. The CSV total is an int (convert the fee strings). The missing file prints a message — never a traceback.

Sample Run

```
Bed 12: stable
Bed 14: observation
Total: 900
No file yet
```

(Your patient names and fees may differ; the **shape** should match.)

Constraints

Use `with open`. UTF-8 for text. `newline=""` for CSV. No logging module yet.

Bonus

If a fee cell is not an int, catch `ValueError` for that row and continue.

Mini-Project (Assignment): Expense Tracker (Files)

Why this matters: Closing Python must not wipe the month. Clerks add daily expenses; the desk needs a CSV Excel can open and a JSON config for categories and a daily limit.

Lab kit: 15_Lab_Assets/01_python_expense_tracker/
Stretch kit: 15_generator_csv_stream — wire `yield` to `csv.DictReader`

This is **not** the in-class assemble. Same tools (`with`, CSV, JSON), fuller ticket. Add `logging` in Logging.

Objective

Record expenses and export a CSV. A bad amount or missing file must print a clear message, not a traceback.

Story / Scenario

Employee Management Portal finance. Clerks add daily expenses. A bad CSV row must not dump a traceback on the desk screen.

Requirements

Piece	Detail
`starter/expense_tracker.py`	Implement `add_expense` and `export_csv`
`data/expenses.csv`	Date, category, amount, note (or the kit's `title,amount,category`)
`data/config.json`	Categories + daily limit (if you extend beyond the starter)
Resilience	Invalid amount / missing file → clear message, no crash
`broken/`	Fix the bugs described in comments
`solution/`	Do **not** copy this

Expected Behavior

- At least three expenses in memory or file
- CSV export opens in Excel / Sheets
- Invalid amount does not crash

Sample Run

Follow the kit README: `python expense_tracker.py` after you implement the TODOs. Seed file: `seed/sample_expenses.csv` (import stretch).

Bonus Challenges

- Enforce daily limit from config (warn or block when exceeded)
- Export a month report as a new CSV
- Stream a large seed file with a generator instead of `readlines()` — kit `15_generator_csv_stream`

Done when

- [ ] Starter kit / your CLI can add expense and show or export totals
- [ ] Bad / missing files handled without traceback for expected cases
- [ ] You did not paste `solution/`

Interview Q&A

Q1. `read()` vs iterating the file object?
`read()` loads the entire file into memory. Iteration reads one line at a time — constant memory for large files.

Q2. What is a context manager?
An object used with `with` that defines `__enter__` and `__exit__` for setup/cleanup. `open()` closes the file in `__exit__`, even if an exception occurred.

Q3. CSV vs JSON — how do you choose?
If the data is a **table** (Excel, import/export, reports) → CSV. If it is **config** or a nested structure (APIs, preferences) → JSON.

Q4. How do you handle a missing file?
`try` / `except FileNotFoundError`, or check the path first; return a default or prompt the user.

Q5. `json.load` vs `json.loads`?
`json.load` reads from a file-like object. `json.loads` parses a JSON **string** already in memory (`s` for string).

Q6. Why is `with open` preferred over manual `close()`?
The context manager closes the file even if an exception occurs, avoiding leaks and ensuring data is flushed. Manual `close()` is easy to skip on error paths.

Q7. When use CSV vs JSON vs a database?
CSV suits flat tabular exports. JSON fits nested config/API payloads. A database is better for concurrent writes, queries, and multi-user integrity — next after this phase's MySQL work.

Q8. `tell()` vs `seek()`?
`tell()` returns the current cursor index (0-based). `seek(offset)` moves the cursor. Text mode in Python 3 seeks from the start; use a value from `tell()` if the offset is not 0.

Q9. `listdir()` vs `walk()`?
`listdir` lists one directory. `walk` visits that directory and all subdirectories.

Q10. What is pickling? When would you not use it?
Pickling serializes a Python object to bytes. Do not unpickle untrusted files. For data you share with other languages or humans, prefer JSON or CSV.

Q11. Why is append mode safer than read-modify-write for an expense log?
Append adds at the end without rewriting the whole file, so a crash is less likely to wipe earlier lines.

Q12. Mode `w` vs method `write()`?
`w` is *how you opened* (create/replace; it truncates). `write()` is *the call* that sends a string. Opening with `w` has already cleared the file.

Cheat Sheet

```
open(path, mode, encoding="utf-8")  → file object
with open(...) as f:                → closes even on error
"r"                                 → read (FileNotFoundError if missing)
"w"                                 → create / replace (truncates)
"a"                                 → append (creates if missing)
f.read()                            → whole file as one string
for line in f:                      → one line at a time (large files)
os.makedirs(path, exist_ok=True)    → create folders
csv.DictWriter / DictReader         → table with header keys; newline=""
json.dump / json.load               → object ↔ file
json.dumps / json.loads             → object ↔ string
```

Text = characters + UTF-8. Binary = bytes (`rb` / `wb`) — no encoding.

Completion Checklist

```
[ ] I can open a text file with with and UTF-8
[ ] I can choose r / w / a and I know w truncates
[ ] I can write, append, and iterate lines
[ ] I can catch FileNotFoundError on a missing path
[ ] I can write and read a CSV with DictWriter / DictReader
[ ] I can save and load a JSON config
[ ] I completed the lab
[ ] I attempted the expense-tracker mini-project
[ ] I can explain CSV vs JSON in an interview
```

What's Next?

You can now keep data after the program closes — text, tables, and config.

Next, Logging writes **what happened** (levels, timestamps, exceptions) to `logs/app.log`. `print` is not enough once the app runs without anyone watching the terminal.